SHOPZO — Project Technical Documentation

Multi-Tenant Retail Management & Point-of-Sale System | Viva Voce Preparation Guide

1. Executive Project Overview

SHOPZO is a modern, high-performance retail management and Point-of-Sale (POS) ecosystem designed for retail businesses, multi-branch stores, and cashiers. Built with an **offline-first mobile architecture** on Android (Kotlin & Jetpack Compose) and backed by a **multi-tenant cloud API server** (Node.js, TypeScript & PostgreSQL), Shopzo enables seamless billing, real-time inventory tracking, customer ledger management (Khata system), and secure staff management.

Component	Technology Stack / Tools	Key Role / Architecture
Frontend Platform	Android Native (Kotlin 1.9+, Target SDK 36)	Mobile POS client UI and offline transaction engine
Frontend UI Framework	Jetpack Compose & Material 3	Declarative reactive UI framework
Mobile Architecture	Clean Architecture + MVVM Pattern	Decoupled layers (UI, ViewModel, Repository, Data)
Local Mobile Database	Android Room Database (SQLite + KSP)	Offline-first data persistence & reactive Flow streams
Async Programming	Kotlin Coroutines & StateFlow	Non-blocking asynchronous execution & reactive UI state
Networking & REST	Retrofit 2 + OkHttp + Kotlinx Serialization	HTTP REST client with JSON serialization & logging
Backend Runtime	Node.js (v18+) & TypeScript (v5.4+)	Type-safe backend API server execution environment
Backend Framework	Express.js (v4.19+)	Lightweight, fast HTTP routing and REST API controllers
Cloud Database	PostgreSQL (v15+) via pg driver	Relational DB with multi-tenant shop_id isolation
Security & Auth	JWT (JSON Web Tokens) & bcryptjs	Stateless authentication & salt password hashing

2. System Architecture & Engineering Highlights

A. Mobile Frontend Architecture (Android Clean Architecture + MVVM)

The Android application is built adhering to Google's official recommended Android Architecture Guidelines. It follows a strict separation of concerns divided into 3 distinct layers:

- **UI / Presentation Layer:** Built completely with **Jetpack Compose** (declarative UI) and Material Design 3. UI states are observed as immutable StateFlow objects, eliminating traditional Android View lifecycle memory leaks.

- **ViewModel Layer:** Acts as the state holder. Handles UI logic, invokes Repository methods via Kotlin Coroutines (viewModelScope), and exposes UI states (e.g., BillingUiState, ProductsUiState).
- **Data & Repository Layer:** Combines the local **Room SQLite Database** and remote **Retrofit Network Client** using the Single Source of Truth (SSOT) pattern for seamless offline-first capability.

B. Backend & Database Architecture

The backend server is implemented in **TypeScript** running on Node.js and Express.js. It provides a robust RESTful API layer for store management, staff administration, product inventory, and data sync.

- **Multi-Tenant Architecture:** Every database query is strictly scoped with a shop_id parameter. This enforces data isolation between different shops sharing the same database infrastructure.
- **Precision Financial Calculation (Integer Paise Pattern):** All monetary values (e.g. buying price, selling price, bill totals) are stored as **64-bit integers in Paise** (1 INR = 100 Paise). This eliminates floating-point precision errors during tax, discount, and total bill calculations.
- **Immutable Billing Snapshots:** Historical bills store snapshot fields (product_name_snapshot, selling_price_paise, buying_price_paise) in bill_items. If a product's name or price changes later, past billing records remain 100% accurate for audit compliance.

3. Relational Database Schema (PostgreSQL)

The backend PostgreSQL database comprises 10 structured tables engineered for scalability, auditing, and high concurrency:

Table Name	Primary Key & Foreign Keys	Core Purpose & Key Fields
users	PK: id	Owner and staff accounts (mobile_number, password_hash)
shops	PK: id, FK: owner_id	Shop profile details (shop_code, name, address)
staff_permissions	PK: id, FK: shop_id, user_id	Role-based access control (role: OWNER/STAFF, permissions JSONB)
categories	PK: id, FK: shop_id	Product categories for inventory organization
products	PK: id, FK: shop_id, category_id	Inventory items (buying_price_paise, selling_price_paise, quantity, min_stock_level)
stock_movements	PK: id, FK: shop_id	Audit log for inventory movements (Types: RESTOCK, ADJUSTMENT, SALE, RETURN)
customers	PK: id, FK: shop_id	Customer ledger & due tracking (total_purchase_paise, outstanding_due_paise)
bills	PK: id, FK: shop_id, customer_id	POS invoices (bill_number, grand_total_paise, paid_amount_paise, payment_status)
bill_items	PK: id, FK: shop_id, bill_id	Line items with price/name snapshots (quantity, selling_price_paise, subtotal_paise)
returns	PK: id, FK: shop_id, bill_id	Item returns & restocking log (quantity_returned, refund_amount_paise, stock_restored)

4. Comprehensive Viva Voce Questions & Model Answers

Below are the top 25 viva questions categorized into key technical domains, complete with precise, high-scoring answers:

Category A: Android & Kotlin Mobile Architecture

Q1: What architecture pattern does the Shopzo Android application use and why?

Answer: Shopzo uses Google's recommended Clean Architecture with the MVVM (Model-View-ViewModel) pattern. This separates the project into UI (Jetpack Compose), ViewModel (business logic & UI state management), and Data (Room DB & Retrofit repository). It makes the codebase modular, highly testable, and prevents lifecycle memory leaks.

Q2: Why did you choose Jetpack Compose over traditional XML Layouts?

Answer: Jetpack Compose is Android's modern declarative UI toolkit. It reduces boilerplate code, eliminates XML parsing overhead, provides reactive UI rendering based on state changes, and allows building dynamic reusable UI components directly in Kotlin.

Q3: How does Shopzo handle offline functionality?

Answer: Shopzo implements an Offline-First approach using Android Room Database. When a transaction or billing operation is performed, data is written locally to Room SQLite first, ensuring immediate UI feedback without relying on internet connectivity. A background worker / sync engine then synchronizes the local changes with the cloud PostgreSQL server when connected.

Q4: What are Kotlin Coroutines and StateFlow, and how are they used in Shopzo?

Answer: Coroutines provide lightweight, asynchronous, non-blocking thread management (e.g. running database queries on Dispatchers.IO without blocking the main UI thread). StateFlow is a state-holder observable flow that emits current and new state updates to the Jetpack Compose UI reactively.

Q5: What is the role of Room KSP (Kotlin Symbol Processing)?

Answer: KSP is a powerful code generation tool by Google that replaces legacy kapt (Annotation Processing Tool). KSP analyzes Kotlin annotations at compile-time to generate Room DAO implementations up to 2x faster than kapt.

Q6: How does Retrofit handle network errors and responses in Shopzo?

Answer: Retrofit converts REST API endpoints into Kotlin interfaces. Combined with OkHttp logging interceptors and Kotlinx Serialization, it handles HTTP status codes, deserializes JSON responses into strongly-typed Kotlin data classes, and catches network timeouts safely.

Q7: What is DataStore Preferences used for in Shopzo?

Answer: DataStore Preferences is used to store lightweight key-value pairs (like user auth tokens, active shop_id, theme settings) asynchronously and transactionally, replacing deprecated SharedPreferences.

Category B: Backend, Node.js & TypeScript API Server

Q8: Why did you choose Node.js and TypeScript for the backend server?

Answer: Node.js provides an asynchronous, non-blocking I/O event loop ideal for handling concurrent REST API requests. TypeScript adds static typing over JavaScript, preventing runtime type errors, improving IDE autocomplete, and ensuring strict API contract consistency.

Q9: What framework is used on the backend and what are its main routes?

Answer: We use Express.js (v4.19). The primary route modules are authRoutes (authentication), shopRoutes (shop management), staffRoutes (role permissions), productRoutes (inventory CRUD), and syncRoutes (cloud sync endpoints).

Q10: How is JWT (JSON Web Token) authentication implemented in Shopzo?

Answer: Upon successful login, the backend signs a JWT payload containing user_id and shop_id using a secret key. Mobile client includes this token in the 'Authorization: Bearer ' header of subsequent API calls. Backend middleware verifies the token before allowing access to protected endpoints.

Q11: How are passwords stored securely in the database?

Answer: Passwords are never stored in plain text. We use bcryptjs to generate a secure cryptographic hash with salt rounds (10-12) before persisting to PostgreSQL.

Q12: What is the purpose of ts-node-dev and tsc in your package.json?

Answer: ts-node-dev is used during development for hot-reloading the server when TypeScript files change. tsc (TypeScript Compiler) compiles TypeScript source code into plain JavaScript inside the dist/ folder for production deployment.

Q13: How does the backend prevent Cross-Origin resource issues and unauthorized access?

Answer: We use the CORS middleware to restrict request origins and custom JWT authorization middleware to validate user identity and shop permission scopes on every route.

Category C: Database Design, PostgreSQL & Multi-Tenancy

Q14: How is Multi-Tenancy implemented in the Shopzo PostgreSQL database?

Answer: Shopzo uses a Shared Database, Shared Schema approach with Tenant Isolation via shop_id columns. Every table (products, customers, bills, staff) contains a shop_id foreign key, and all SQL queries strictly include WHERE shop_id = $1 to ensure complete data separation between stores.

Q15: What is the Integer Paise Pattern and why is it critical for billing software?

Answer: Floating-point numbers (float/double) in programming suffer from binary representation inaccuracies (e.g. 0.1 + 0.2 = 0.30000000000000004). In Shopzo, all financial fields are stored as integer Paise (1 INR = 100 Paise). This guarantees 100% exact currency arithmetic without rounding errors.

Q16: Explain the Billing Snapshot Pattern used in bill_items.

Answer: When a bill is generated, product details (name, buying price, selling price) are copied as snapshots into the bill_items table. If the store owner later renames the product or changes its selling price in inventory, historical bills remain unmodified, maintaining accurate financial records.

Q17: How does Shopzo maintain an audit trail for stock inventory?

Answer: Inventory changes are tracked using the stock_movements table. Any restock, sale, return, or manual adjustment creates an immutable movement record with product_id, quantity change, movement type, timestamp, and optional reason.

Q18: Why did you choose PostgreSQL over MongoDB (NoSQL)?

Answer: Retail POS billing requires strict ACID transactions, complex relational joins (e.g., Bills -> Bill Items -> Products -> Customers), foreign key integrity constraints, and exact structured queries, for which PostgreSQL is significantly superior to NoSQL databases.

Q19: What database indexes were created and why?

Answer: Indexes were added on high-frequency query columns like idx_products_shop (products.shop_id), idx_bills_shop (bills.shop_id), and idx_users_mobile (users.mobile_number) to ensure sub-millisecond query execution even as data grows.

Category D: Business Logic, Security & Offline Sync

Q20: How does the Staff Permission System work?

Answer: Shopzo supports OWNER and STAFF roles. The staff_permissions table stores fine-grained JSON permissions for staff members (e.g., allow billing, restrict price editing, restrict stock modification), which are checked by both mobile UI and backend middleware.

Q21: How are customer dues and Khata ledgers managed?

Answer: Each customer record tracks total_purchase_paise and outstanding_due_paise. When a partial payment or credit bill is issued, pending_amount_paise is added to the customer's due ledger. Subsequent payment receipts decrease the outstanding due.

Q22: Explain the item return and restocking workflow.

Answer: When a return is processed, the system records the returned quantity and refund amount in the returns table. If stock_restored is set to true, the product's inventory quantity is automatically incremented and a RESTOCK movement is logged.

Q23: How are primary key IDs generated across offline clients?

Answer: Shopzo uses UUID v4 (Universally Unique Identifiers) for primary keys generated on the mobile app. Because UUIDs are globally unique, multiple mobile clients can create records offline simultaneously without primary key collision during sync.

Q24: What happens if a customer attempts a transaction offline?

Answer: The transaction is saved immediately to the local Room database, assigned an offline status, and added to the sync queue. Once network connectivity is restored, the sync manager pushes queued transactions to syncRoutes on the backend.

Q25: What are the main future enhancements planned for Shopzo?

Answer: Future enhancements include AI-powered sales forecasting, barcode scanner hardware integration via camera, thermal Bluetooth printer receipt printing, and automated WhatsApp bill sharing.